

OSSP (Operating Systems And Systems Programming) - 25CS2104E

PRACTICAL 3

Objective: Create a parent and child using fork(), display PID/PPID, and observe process states with ps, top and /proc. This follows Practical 3 of the uploaded document.

Step 1 — Open Ubuntu Terminal

Press Ctrl + Alt + T.

Ctrl + Alt + T

You should see something similar to:

A terminal window opens with a shell prompt such as: user@ubuntu:~\$

Step 2 — Create the Practical Folder

Create a separate folder and enter it.

```
mkdir -p ~/ossp_practical3
cd ~/ossp_practical3
pwd
```

Step 3 — Create the C Program

Open a source file.

nano process_states.c

You should see something similar to:

GNU nano opens.

Paste this exact program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent PID: %d\n", getpid());
    printf("Parent PPID: %d\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
}
```

```

}

if (pid == 0) {
    printf("Child PID: %d\n", getpid());
    printf("Child PPID: %d\n", getppid());
    printf("Child: Running\n");
    sleep(5);
    printf("Child: Terminating\n");
    exit(0);
}

printf("Parent: Child PID = %d\n", pid);
printf("Parent: Waiting for child...\n");
wait(NULL);
printf("Parent: Child completed\n");

return 0;
}

```

Save: Ctrl + O → Enter → Ctrl + X

Step 4 — Compile the Program

Compile using GCC.

```
gcc process_states.c -o process_states
```

No error message normally means compilation succeeded.

Step 5 — Run the Program

Run the program and observe the parent and child information.

```
./process_states
```

Your PID and PPID numbers will be different. The child's PPID should match the parent's PID.

Step 6 — Understand fork()

fork() creates a new child process.

```
pid = fork();
```

You should see something similar to:

In the child: fork() returns 0.

In the parent: fork() returns the child's PID.

If fork() returns a negative value, creation failed.

Step 7 — Observe Processes with ps

Run the program in the background so it remains available for inspection.

```
./process_states &
ps -o pid,ppid,stat,cmd | grep process_states
```

Step 8 — Understand the Process State

The STAT column shows the current process state.

```
ps -o pid,ppid,stat,cmd -p 12000,12001
```

S commonly indicates sleeping/waiting. The exact state can change depending on timing.

Step 9 — Observe with top

Use top for live process monitoring.

```
top
```

Press q to exit top.

Step 10 — Inspect /proc

Linux exposes information about every running process under /proc/PID.

```
cat /proc/12001/status | head -20
```

Replace 12001 with your actual child PID.

Step 11 — Observe the Four States

Use the monitoring tools during the program execution.

```
ps -o pid,ppid,stat,cmd  
cat /proc/PID/status  
top
```

You should see something similar to:

Ready: process is eligible to run but is waiting for CPU scheduling.

Running: process is currently executing.

Waiting: process is sleeping or waiting for an event.

Terminated: process has finished and exited.

Step 12 — Record the Parent–Child Flow

Write the observed execution sequence in your record.

You should see something similar to:

```
Parent created  
↓  
fork()  
↙ ↘  
Parent  Child  
↓      ↓  
wait()  Running  
↓      ↓  
Waiting sleep()  
↓      ↓  
Child finishes  
↓  
Parent resumes  
↓  
Parent terminates
```

Step 13 — Final Test

Run the program again and record the actual PID values.

```
./process_states
```

FINAL RESULT: The parent and child are created with fork(), PID/PPID values are displayed, wait() synchronizes them, and process states are investigated using ps, top and /proc.

